



QEMU + LLVM:

**less complexity,
better performance**

rev.ng Labs

rev.ng Labs



1. 10 people
2. Partly in Milan, Italy, partly in rest of Europe
3. Compiler engineers/security researchers
4. Customers

Qualcomm

HUAWEI

Outline

1. Brief QEMU intro
2. helper2tcg: from interpreter to JIT translator
3. Demo: helper2tcg on xqciu
4. Generating test cases with KLEE
5. What's next: using LLVM JIT in QEMU

The QEMU emulator

- High performance thanks to JIT compilation
- Instruction-accurate (not clock accurate)
- Open source, very active, de-facto standard
- Also does virtualization

Interpreter vs JIT

For each instruction you can:

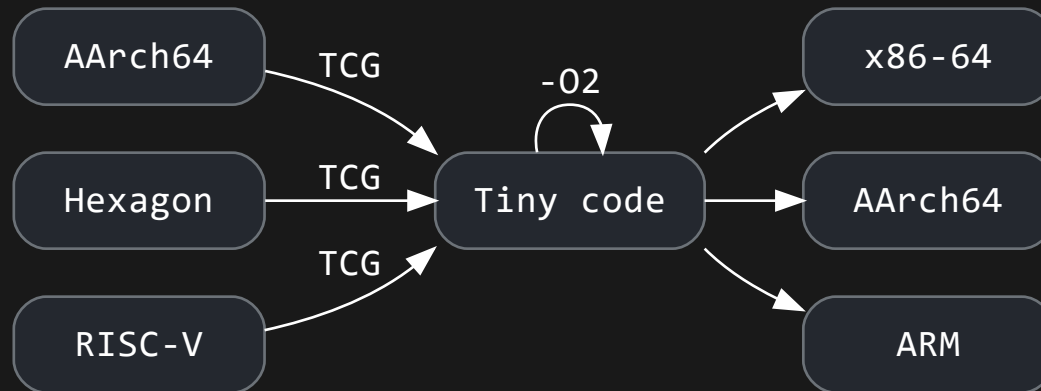
1. Implement it in C, i.e., an **helper function**, or
2. Write a **tiny code generator**

Helper functions

```
int32_t HELPER(A2_add)(CPUHexagonState *env,  
                      int32_t RsV, int32_t RtV)  
{  
    int32_t RdV = 0;  
    RdV=RsV+RtV;  
    return RdV;  
}
```

- Pro: easy to implement
- Pro: typically already available
- Con: bad performance

QEMU architecture



TCG example

```
void A2_add(TCGv_i32 ret, TCGv_env env,  
            TCGv_i32 v_0, TCGv_i32 v_1)  
{  
    TCGv_i32 add_2 = tcg_temp_new_i32();  
    tcg_gen_add_i32(add_2, v_1, v_0);  
    tcg_gen_mov_i32(ret, add_2);  
}
```

- Pro: very fast
- Con: requires deep QEMU knowledge
- Con: error-prone

helper2tcg

Can we turn an helper function into TCG?

helper2tcg: main idea

C

helper2tcg: main idea

C

1. Compile helper with clang

helper2tcg: main idea

C LLVM IR

1. Compile helper with clang
2. Obtain LLVM's internal representation

helper2tcg: main idea

C LLVM IR

1. Compile helper with clang
2. Obtain LLVM's internal representation
3. Optimize it

helper2tcg: main idea

C LLVM IR $\xrightarrow{\text{helper2tcg}}$ TCG

1. Compile helper with clang
2. Obtain LLVM's internal representation
3. Optimize it
4. Translate it into a TCG

helper2tcg: main idea

C LLVM IR $\xrightarrow{\text{helper2tcg}}$ TCG

1. Compile helper with clang
2. Obtain LLVM's internal representation
3. Optimize it
4. Translate it into a TCG
5. Merge it into QEMU

End result

Simplicity of writing an helper.

Performance of TCG.

helper2tcg: main idea

C ^{clang} $\longrightarrow$ LLVM IR ^{helper2tcg} $\longrightarrow$ TCG

```
int32_t HELPER(A2_paddt)(CPUHexagonState *env,  
                        int32_t RdV, int32_t PuV,  
                        int32_t RsV, int32_t RtV)  
{  
    {if (fLSBOLD(PuV)) {RdV=RsV+RtV;} else {CANCEL;}}  
    return RdV;  
}
```


helper2tcg: main idea



```
define i32 @helper_A2_paddt(%struct.CPUArchState* %0,  
                           i32 %1, i32 %2,  
                           i32 %3, i32 %4) {  
    %7 = and i32 %2, 1  
    %.not = icmp eq i32 %7, 0  
    %8 = add i32 %4, %3  
    %spec.select = select i1 %.not, i32 %1, i32 %8  
    ret i32 %spec.select  
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: main idea

clang
C $\longrightarrow$ LLVM IR $\longrightarrow$ **helper2tcg** **TCG**

```
void emit_A2_paddt(TCGv_i32 ret, TCGv_env env,
                  TCGv_i32 v_0, TCGv_i32 v_1,
                  TCGv_i32 v_2, TCGv_i32 v_3) {
    TCGv_i32 and_5 = tcg_temp_new_i32();
    tcg_gen_andi_i32(and_5, v_1, 1);
    TCGv_i32 icmp_6 = tcg_temp_new_i32();
    tcg_gen_setcondi_i32(TCG_COND_EQ, icmp_6, and_5, 0);
    TCGv_i32 add_7 = tcg_temp_new_i32();
    tcg_gen_add_i32(add_7, v_3, v_2);
    TCGv_i32 select_8 = tcg_temp_new_i32();
    TCGv_i32 vc_9 = tcg_constant_i32((uint64_t)0ULL);
    tcg_gen_movcond_i32(TCG_COND_NE, select_8, icmp_6, vc_9,
    tcg_gen_mov_i32(ret, select_8);
}
```


helper2tcg: vector instructions

```
void HELPER(V6_vaddhsat) (CPUHexagonState *env,  
                          void * restrict VdV_void,  
                          void * restrict VuV_void,  
                          void * restrict VvV_void)  
{  
    uint32_t slot __attribute__((unused)) = 4;  
    {  
        fHIDE(int i;)  
        fVFOREACH(16, i)  
        {  
            VdV.h[i] = fVSADDSAT(16, VuV.h[i], VvV.h[i]);  
        }  
    }  
}
```


helper2tcg: vector instructions

```
void emit_V6_vaddhsat(TCGv_env env,  
                      intptr_t vo_0,  
                      intptr_t vo_1,  
                      intptr_t vo_2) {  
    tcg_gen_gvec_ssadd(MO_16, vo_0, vo_2, vo_1, 128, 128);  
}
```


Status

- Presented at QEMU's conf (KVM forum)
- Instruction count: 1408/2160
- Almost ready for upstream
- Supports LLVM 10-14
 - LLVM 15+ planned post upstream

Performance

1. General purpose integer benchmarks
2. Hexagon vector heavy binaries (instruction count)
3. Hexagon vector instructions microbenchmarks

General purpose integer benchmarks

Benchmark	Speedup
coremark	2.43x
stream	1.60x
nbench	3.09x

Speedup is w.r.t. helper-based implementation

See: <https://github.com/nfinit/ansibench>

Instruction count reduction

Binary	Executed inst. reduction (%)
nn tests	9.6%
corner det.	11%

Hexagon vector instructions

- Geometric mean: 4.03x
- Best case: 33.6x speedup
- Worst case: 1.2x slowdown

Can we use **helper2tcg**
on Qualcomm
RISC-V extensions?

Demo time

Questions?

Extra goodies: generating test cases

Given the C function,
can we generate test cases **automatically**?

KLEE

- Symbolic execution engine for LLVM IR
- Automatically generates interesting test cases
- Maximizes test of corner cases/coverage

addsat

```
void qc32_addsat(uint8_t rs1, uint8_t rs2, uint8_t rd) {
    XReg sum = X[rs1] + X[rs2];
    XReg most_negative_number = 1 << (xlen() - 1);
    XReg most_positive_number = (1 << (xlen() - 1)) - 1;
    // overflow occurs if the operands are the same sign and
    if (X[rs1][xlen()-1] == X[rs2][xlen()-1]) {
        if (sum[xlen()-1] != X[rs1][xlen()-1]) {
            if (_signed(X[rs1]) < 0) {
                sum = most_negative_number;
            } else {
                sum = most_positive_number;
            }
        }
    }
    // otherwise, overflow did not occur
    X[rd] = sum;
}
```

test	rs1	rs2	rd
1	-2139062144	0	-2139062144
2	0	0	0
3	2147483647	128	2147483647
4	-2147483617	-2147483648	-2147483648

Questions?

What's next for QEMU?

LLVM as JIT engine

The **tiny** code optimizer

QEMU features a small optimization engine that:

- Grabs low-hanging fruits
- Keeps latency low
- Optimizes individual basic blocks only
- Features no aggressive optimizations

Multi-tier JIT optimizers

- JVM and V8 have multiple optimization tiers
- They use aggressive optimizers for hot code

Can we use LLVM
as a tier-2 optimizer
for QEMU?

Opportunities

- Use LLVM, very mature, JIT infra: ORC
- Exploit LLVM's aggressive optimizations
- Optimize multiple basic blocks
- Specifically, we could optimize loops!

Challenges

- LLVM can be slow, we need to:
 1. limit its usage to hot paths
 2. profile code to identify hot paths
 3. cache optimized code on disk
- Invalidation in case of self-modifying code
- Upstreaming

Related works

- HQEMU (and others)
- Us! We use LLVM in a mainframe emulator.
 - 2x speedup w.r.t. existing JIT
 - Efficient handling of self-modifying code
 - Relocatable code + on-disk cache
 - Overall, we have significant insights

Other directions

- We're working on heterogeneous TCG
- Hard-floats in QEMU?

Recap

We can work on:

1. helper2tcg for RISC-V extensions
2. Generating test cases automatically
3. LLVM as a tier-2 JIT engine in QEMU
4. Heterogeneous TCG
5. Hard-floats in QEMU

Thanks for the attention!

Questions?

